

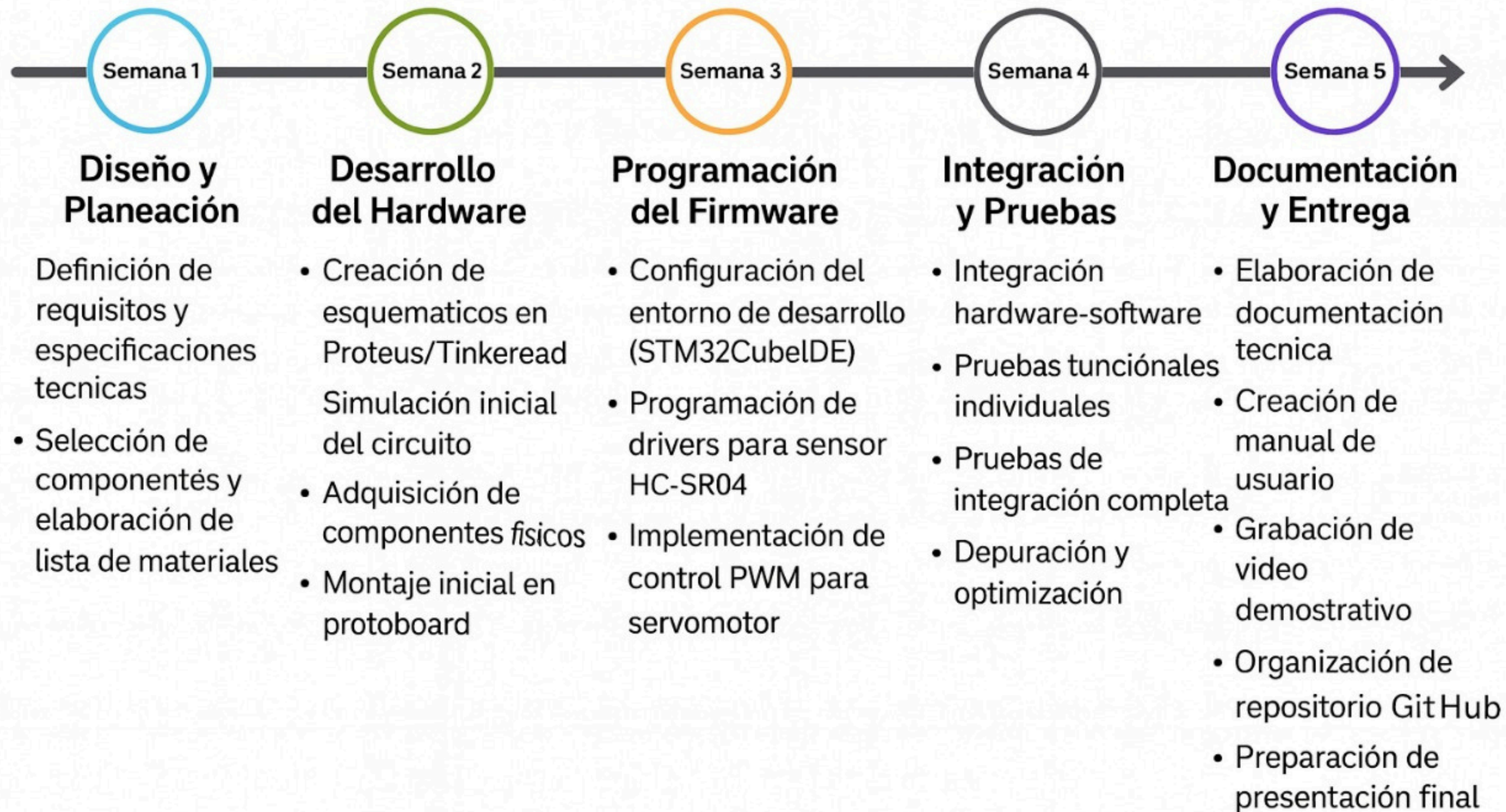
Control de Acceso Vehicular

Diego Eduardo Hurtado Hernandez
Daniel Martinez Sanchez
Jose Salvador Montes Luna
Yael Olvera Mora
Luis Roberto Juarez Manzo



Cronograma y Plan de Trabajo

Cronograma y Plan de Trabajo



Estructura del Proyecto

```
1 from machine import Pin
2 import time
3 import network
4 import socket
5
6 # ... (Bloque de impresión inicial) ...
7
8 # ===== CONFIGURA WIFI =====
9 WIFI_SSID = "INFINITUMA0E4"
10 WIFI_PASSWORD = "TuPassword"
11 # =====
12
13 # ===== HARDWARE =====
14 # LEDs
15 LED_VERDE = Pin(3, Pin.OUT) # GP3, Pin 5
16 LED_ROJO = Pin(4, Pin.OUT) # GP4, Pin 6
17 LED_PICO = Pin("LED", Pin.OUT) # LED interno azul
18
19 # Sensor ultrasónico
20 TRIG = Pin(0, Pin.OUT) # GP0, Pin 1
21 ECHO = Pin(1, Pin.IN) # GP1, Pin 2
22
23 # ===== VARIABLES =====
24 ip_address = "Sin conexión"
25 estado_sistema = "● CERRADO"
26 modo_control = "auto" # auto, manual, telefono
27 ultima_conexion = 0
```

1.- Inicialización y Configuración

Esta sección define las bibliotecas necesarias, configura la red Wi-Fi y establece los pines GPIO para los componentes de hardware.

- Bibliotecas: Se importan Pin, time, network, y socket para control de pines, temporización, conexión Wi-Fi y servidor web.
- Wi-Fi: Se establecen el SSID (WIFI_SSID) y la contraseña (WIFI_PASSWORD).
- Hardware: Se asignan pines a los LEDs (Rojo, Verde, y el LED interno del Pico W) y al sensor ultrasónico (pines TRIG y ECHO).

Estructura del Proyecto

```
1 # ===== FUNCIONES SENSOR =====
2 def medir_distancia():
3     # ... (Código para medir distancia) ...
4     distancia = (fin - inicio) * 0.0343 / 2
5     return distancia if 2 < distancia < 400 else -1
6
7 def actualizar_leds(estado):
8     """Controlar LEDs según estado"""
9     if estado == "cerrado":
10         LED_ROJO.value(1)
11         LED_VERDE.value(0)
12         LED_PICO.value(0)
13     elif estado == "abierto":
14         # ...
15     elif estado == "alerta":
16         # ...
17     elif estado == "transicion":
18         # ...
```

2.-Funciones del Sensor y LEDs

Estas funciones manejan la lógica de bajo nivel para medir la distancia con el sensor y actualizar el estado visual con los LEDs.

Medición de Distancia (medir_distancia)
Implementa el algoritmo del sensor ultrasónico HC-SR04 (tiempo de vuelo):

- 1.- Genera un pulso corto en el pin TRIG para iniciar la medición.
 - 2.- Mide la duración del pulso de retorno en el pin ECHO.
 - 3.- Calcula la distancia usando la fórmula: $\text{Distancia} = (\text{Tiempo de pulso}) \times (\text{Velocidad del sonido}) / 2$
- Punto Clave: El código usa la constante 0.0343/2

Estructura del Proyecto

```
1 # ===== FUNCIONES WiFi =====
2 def conectar_wifi():
3     """Conectar a red WiFi y obtener IP"""
4     global ip_address
5
6     # ... (Proceso de conexión) ...
7
8     if wlan.isconnected():
9         ip_address = wlan.ifconfig()[0]
10        print(f" CONEXIÓN EXITOSA!")
11        print(f" IP: {ip_address}")
12        return True
13    else:
14        print(" NO SE PUDO CONECTAR")
15        # ... (Mensajes de error) ...
16        return False
```

3.- Función de Conexión Wi-Fi:

Esta función establece la conexión a la red Wi-Fi configurada.

- Intenta conectar el Pico W (network.STA_IF) usando el SSID y la contraseña.
- Espera un máximo de 10 segundos para la conexión (20 iteraciones de 0.5s).
- Si la conexión es exitosa, guarda la dirección IP del dispositivo.

Estructura del Proyecto

4.- Interfaz Web

La función `crear_pagina_principal` genera el código HTML completo con estilos CSS para mostrar la interfaz de control en un dispositivo móvil.

- Muestra el estado actual del sistema (Cerrado, Abierto, Transición) basado en el estado de los LEDs.
- Muestra la lectura del sensor (estado_sensor) y la distancia en cm.
- Incluye tres botones para enviar comandos al Pico W:
 - ABRIR ACCESO (/abrir)
 - CERRAR ACCESO (/cerrar)
 - MODO AUTOMÁTICO (/auto)

```
1 # ===== INTERFAZ WEB MEJORADA =====
2 def crear_pagina_principal():
3     """Crear página web para control"""
4     distancia = medir_distancia()
5
6     # ... (Lógica para determinar estado_sensor y estado_leds) ...
7
8     return f"""<!DOCTYPE html>
9 <html lang="es">
10 <head>
11     </head>
12 <body>
13     <script>
14         // ... (Funciones JavaScript para enviar comandos y recargar) ...
15     </script>
16 </body>
17 </html>"""
```


Estructura del Proyecto

```
1 # ===== MANEJO DE CONEXIONES =====
2 def manejar_cliente(cliente):
3     # ... (Lógica para procesar /abrir, /cerrar, /auto, /estado o servir la página principal) ...
4
5 def modo_automatico_silencioso():
6     # ... (Lógica para abrir/cerrar basado en distancia < 15cm y > 20cm) ...
7
8 # ===== SERVIDOR WEB SILENCIOSO =====
9 def iniciar_servidor_silencioso():
10     # ... (Configuración y loop principal del servidor) ...
11     try:
12         while True:
13             try:
14                 cliente, addr = server.accept()
15                 # ... manejar_cliente(cliente) ...
16             except OSError as e:
17                 # Si hay timeout, ejecutar modo automático
18                 if modo_control == "auto":
19                     modo_automatico_silencioso()
```

5.- Lógica del Sistema y Servidor

Esta es la sección central que maneja la interacción con los clientes y la lógica del modo automático.

Manejo de Cliente (manejar_cliente)

- Recibe la solicitud HTTP del cliente.
- Analiza la URL solicitada (p. ej., GET /abrir, GET /cerrar, GET /auto).
- Si detecta un comando, actualiza el estado de los LEDs (actualizar_leds) y el modo_control.
- Si no hay un comando, envía la página web principal generada por crear_pagina_principal.
- Envía la respuesta HTTP y cierra la conexión.

Estructura del Proyecto

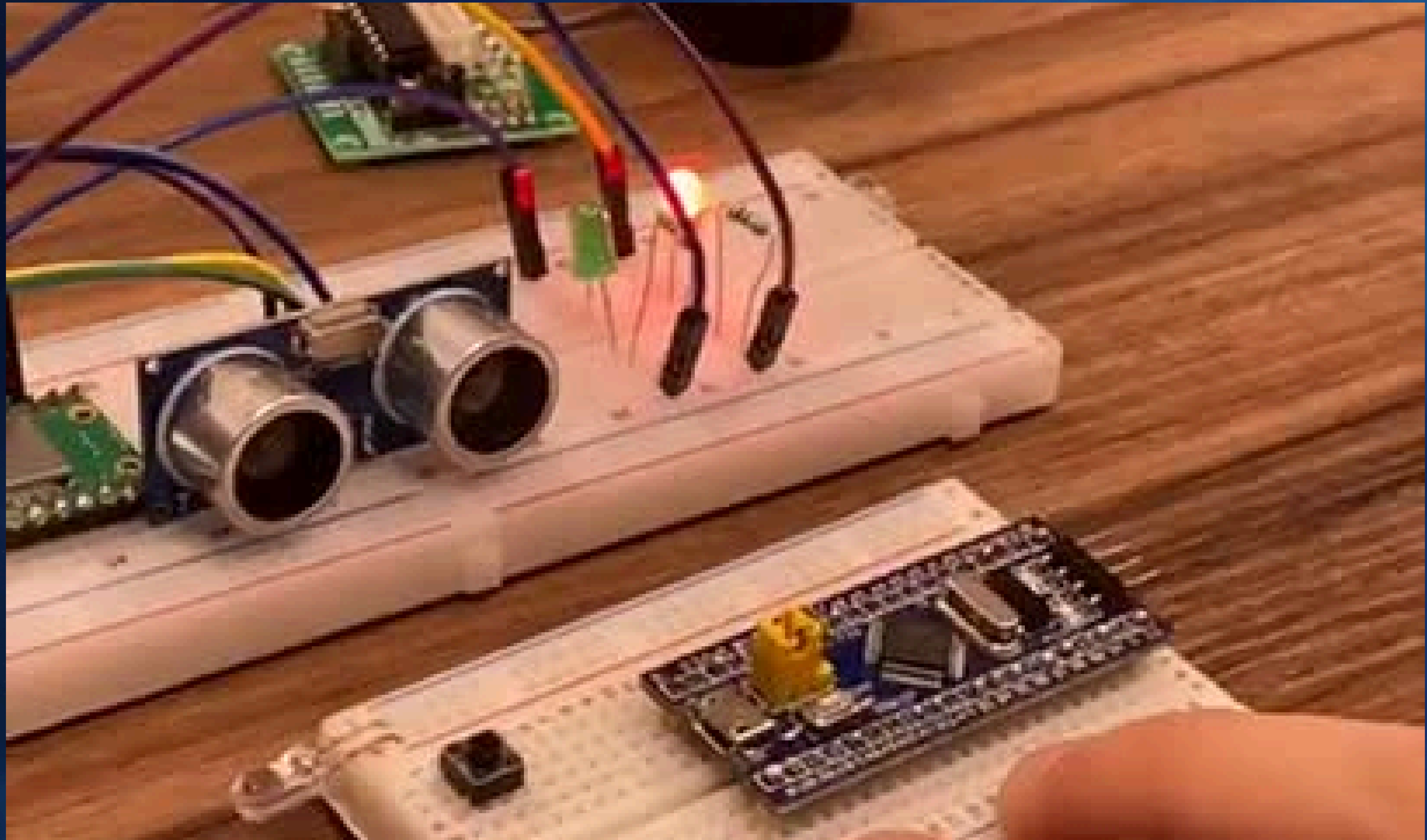
```
1 # ===== PROGRAMA PRINCIPAL =====
2 def main():
3
4     # ... (Inicialización de LEDs y mensajes) ...
5
6     # Conectar WiFi
7     if conectar_wifi():
8         # Iniciar servidor silencioso
9         iniciar_servidor_silencioso()
10    else:
11        # Modo local sin WiFi
12        while True:
13            modo_automatico_silencioso()
14            time.sleep(0.5)
15
16 # ===== EJECUTAR =====
17 if __name__ == "__main__":
18     # ... (Manejo de excepciones y ejecución de main) ...
```

6.- Programa Principal

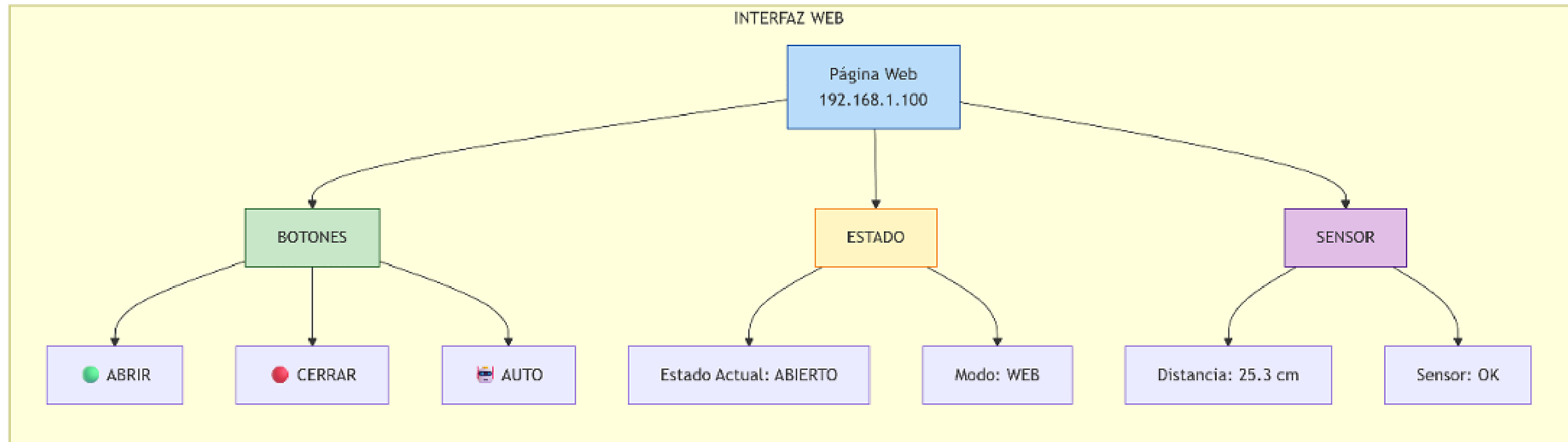
La función main() organiza el flujo del programa.

1. Inicialización: Establece el estado inicial de los LEDs.
2. Conexión: Llama a conectar_wifi().
3. Ejecución:
 - Si el Wi-Fi conecta, inicia iniciar_servidor_silencioso().
 - Si falla el Wi-Fi, entra en un loop de modo local básico, ejecutando solo modo_automatico_silencioso() sin la interfaz web.

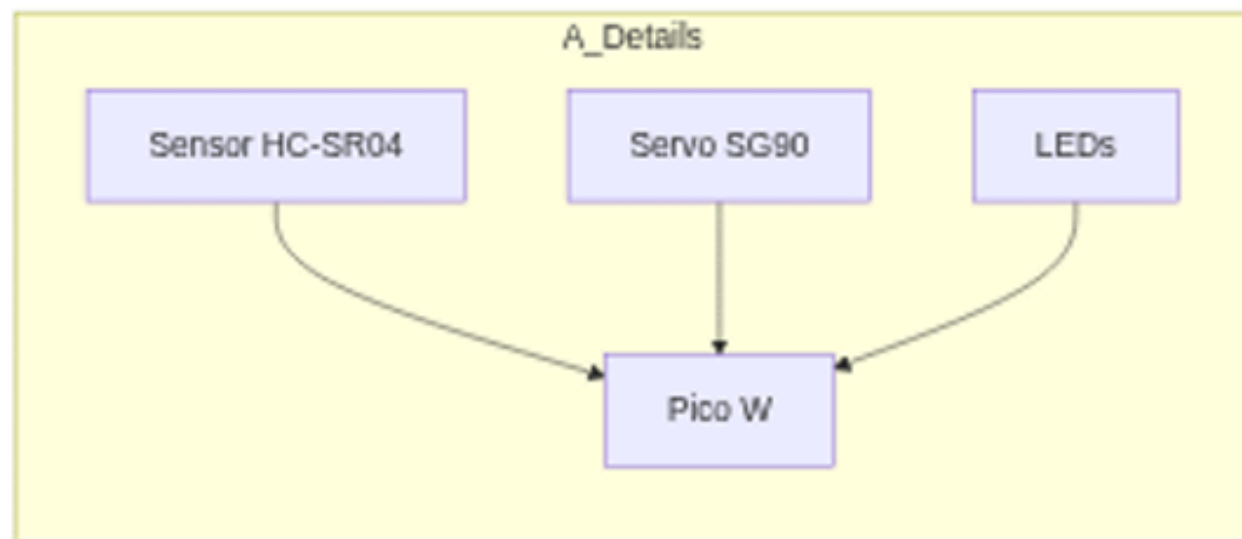
Video del Funcionamiento



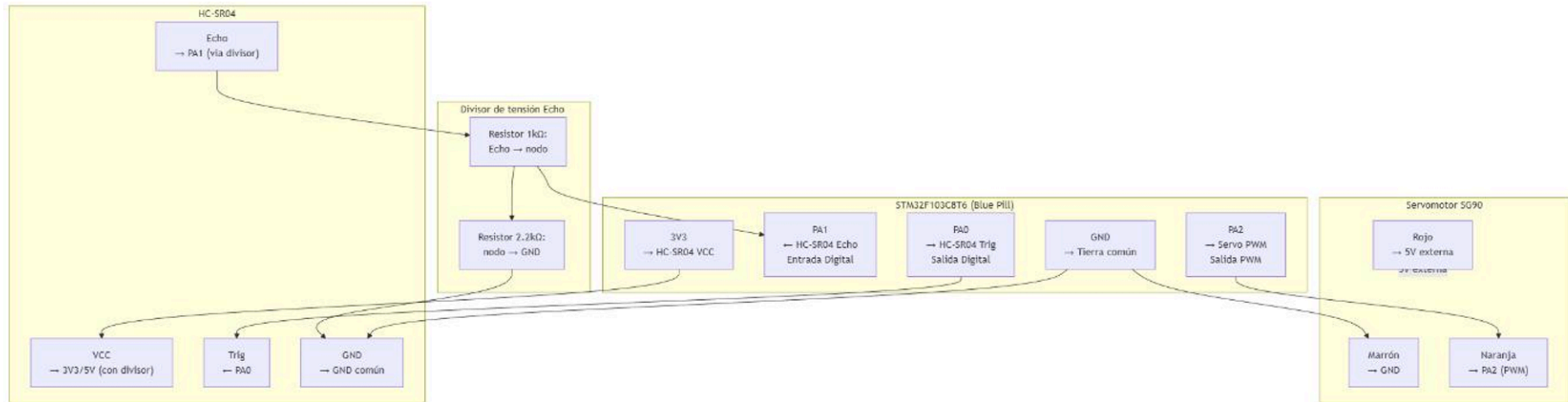
Interfaz Web



La Guia de Instalacion Rapida



Esquemático del Circuito – Raspberry



Arquitectura del Sistema

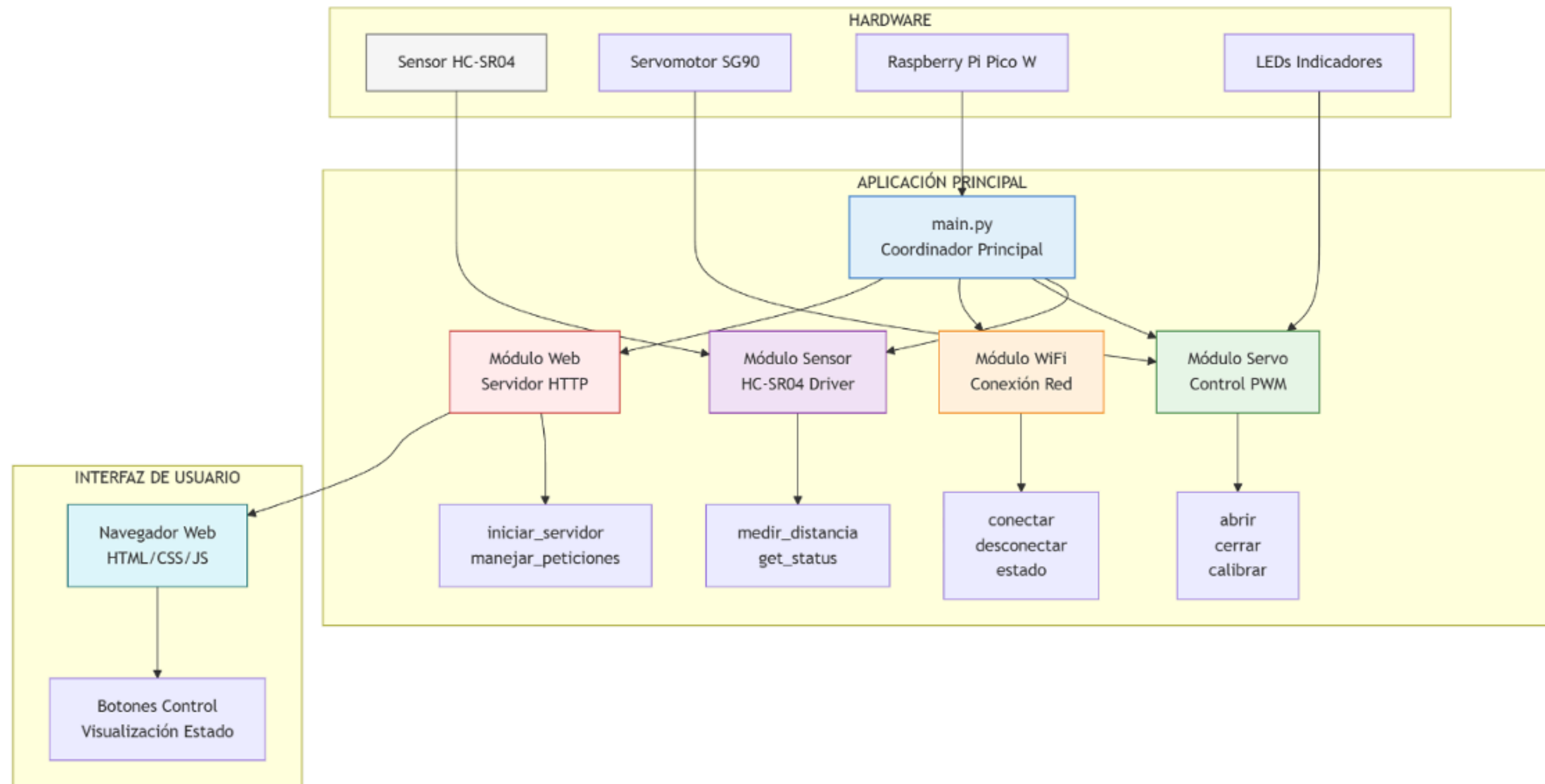
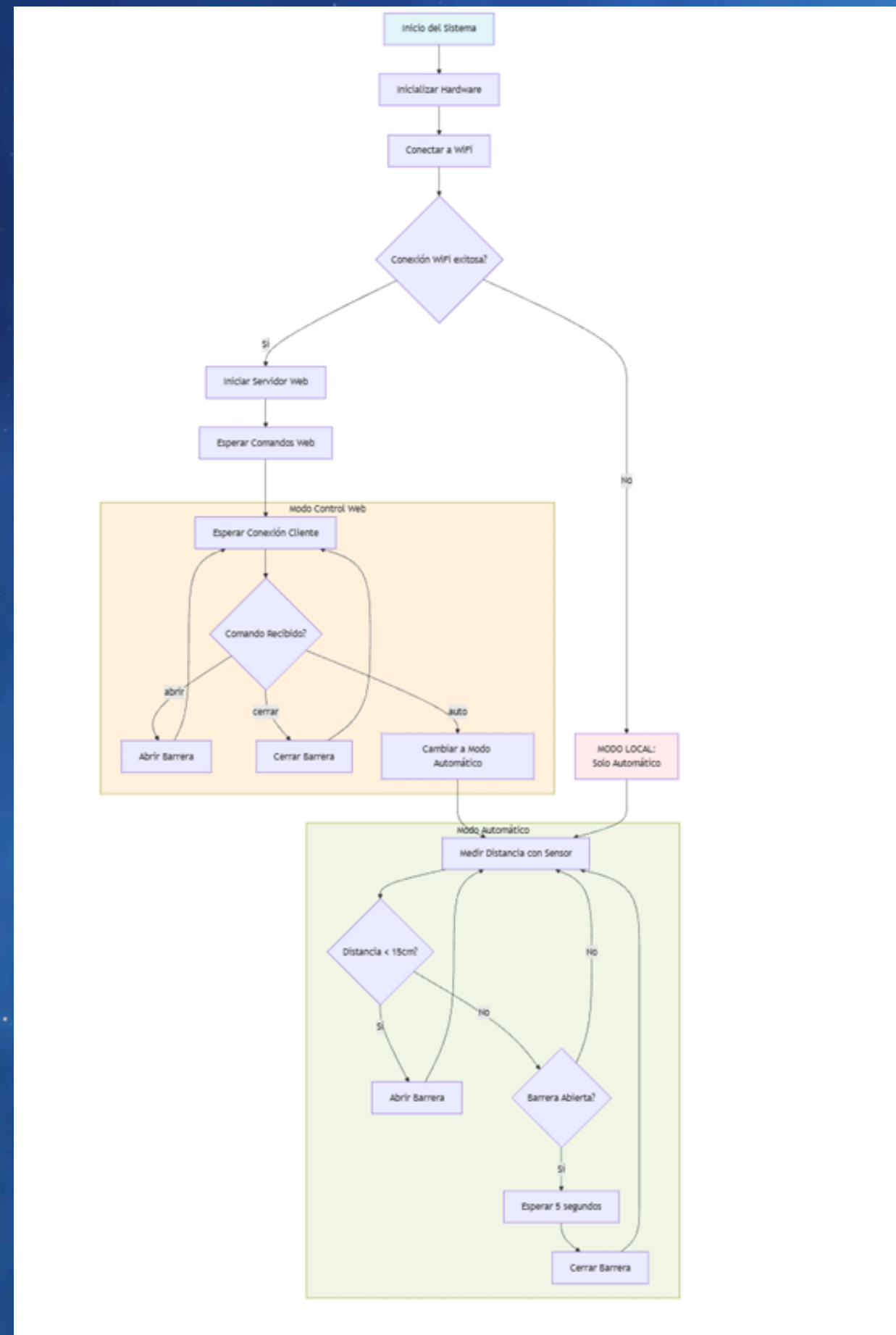
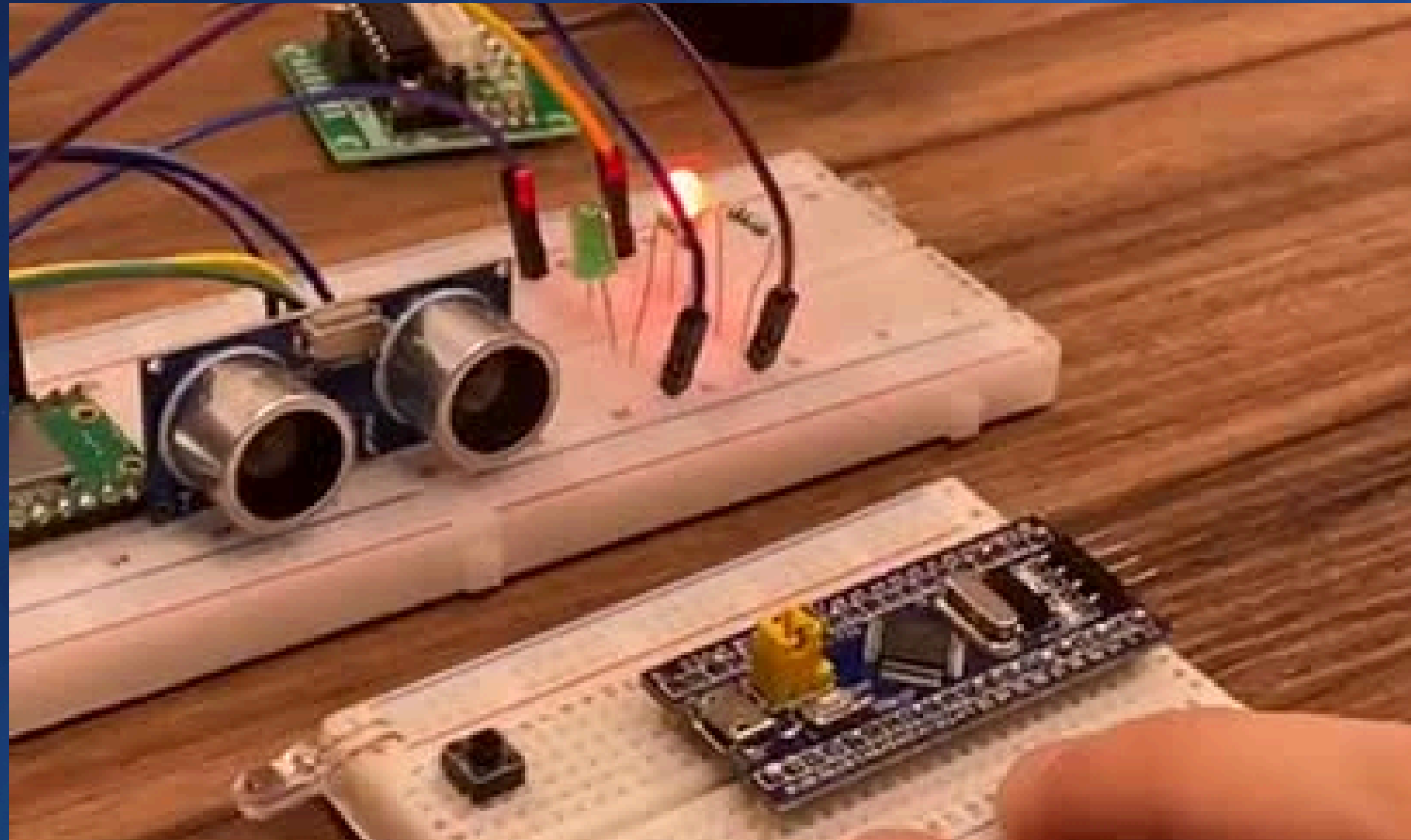


Diagrama de Flujo del Programa



Enlace del Video

<https://www.youtube.com/shorts/dBVKEIYAGLI>



**Muchas Gracias Por
su Atencion**